

Examples for HW 8 Helper Functions

Before testing, type in your last name

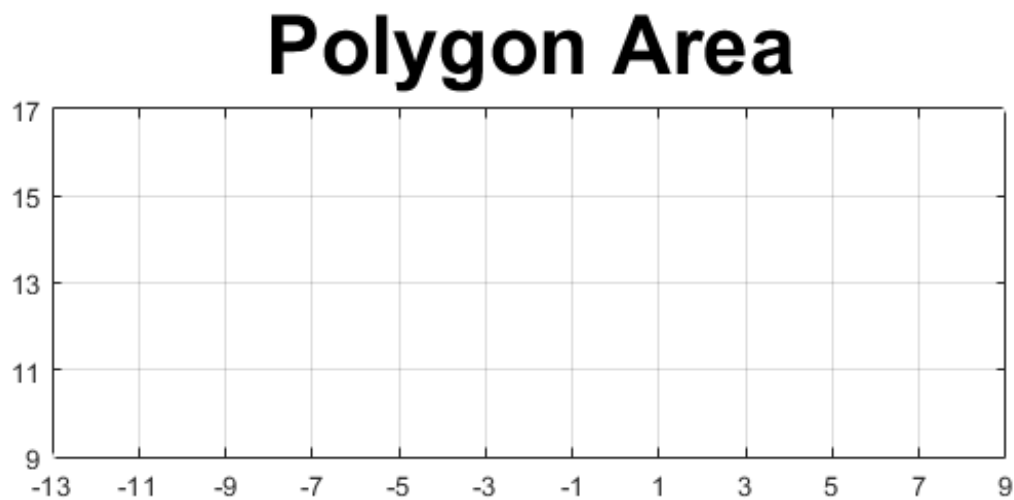
```
lastname = "Key";
```

HW8P1_initial_display

```
program_name = "HW8P1_initial_display_";  
calling_function = str2func(program_name + lastname);
```

Example 1

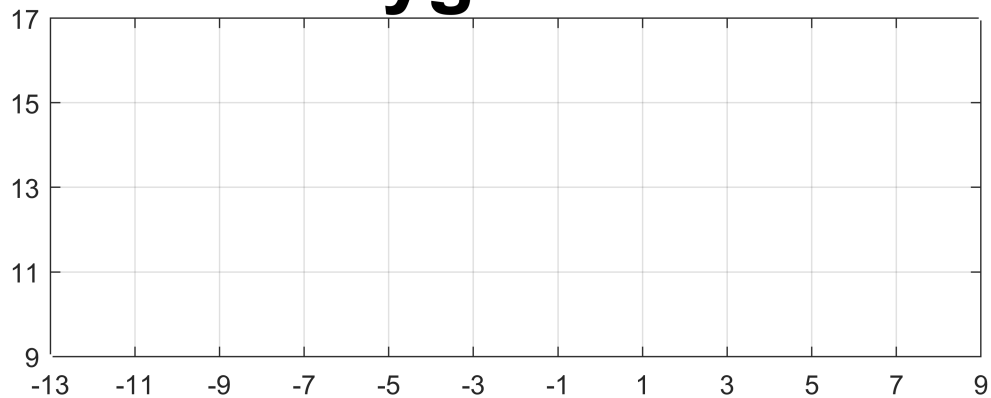
A correct output will look like the following:



```
calling_function(-13,9,9,17)
```

```
hold off;
```

Polygon Area



HW8P2_is_pt_on_line_segment

```
program_name = "HW8P2_is_pt_on_line_segment_";  
calling_function = str2func(program_name + lastname);
```

Example 1

correct output: false

```
line = struct('x1',0,'y1',0,'x2',5,'y2',5);  
is_on_line = calling_function(3,2,line)
```

```
is_on_line = logical  
            0
```

Example 2

correct output: true

```
line = struct('x1',0,'y1',0,'x2',5,'y2',5);
is_on_line = HW8P2_is_pt_on_line_segment_Key(3,3,line)

is_on_line = logical
    1
```

Example 3

correct output: false

```
line = struct('x1',0,'y1',0,'x2',5,'y2',5);
is_on_line = HW8P2_is_pt_on_line_segment_Key(0,0,line)

is_on_line = logical
    0
```

HW8P3_do_line_segments_intersect

```
program_name = "HW8P3_do_line_segments_intersect_";
calling_function = str2func(program_name + lastname);
```

Example 1

correct output: true

```
line1 = struct('x1',0,'y1',0,'x2',5,'y2',5);
line2 = struct('x1',0,'y1',7,'x2',3,'y2',0);

% For a visual of the lines uncomment the code below
% figure
% hold on;
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')
% hold off;
intersect = calling_function(line1,line2)

intersect = logical
    1
```

Example 2

correct output: false

```
line1 = struct('x1',0,'y1',0,'x2',5,'y2',5);
line2 = struct('x1',9,'y1',7,'x2',3,'y2',0);

% For a visual of the lines uncomment the code below
% figure
% hold on;
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')
% hold off;
```

```
intersect = calling_function(line1,line2)
```

```
intersect = logical  
0
```

Example 3

correct output: false

```
line1 = struct('x1',0,'y1',2,'x2',4,'y2',2);  
line2 = struct('x1',3,'y1',0,'x2',6,'y2',6);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% hold off;  
intersect = calling_function(line1,line2)
```

```
intersect = logical  
0
```

Example 4

correct output: true

```
line1 = struct('x1',0,'y1',2,'x2',6,'y2',2);  
line2 = struct('x1',3,'y1',0,'x2',6,'y2',6);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% hold off;  
intersect = calling_function(line1,line2)
```

```
intersect = logical  
1
```

Example 5

correct output: true

```
line1 = struct('x1',0,'y1',2,'x2',4,'y2',2);  
line2 = struct('x1',3,'y1',0,'x2',3,'y2',6);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% hold off;
```

```
intersect = calling_function(line1,line2)
```

```
intersect = logical  
1
```

Example 6

correct output: false

```
line1 = struct('x1',1,'y1',0,'x2',1,'y2',6);  
line2 = struct('x1',3,'y1',0,'x2',3,'y2',6);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% axis([-1 4 -1 8])  
% hold off;  
intersect = calling_function(line1,line2)
```

```
intersect = logical  
0
```

Example 7

correct output: false

```
line1 = struct('x1',0,'y1',0,'x2',1,'y2',1);  
line2 = struct('x1',2,'y1',2,'x2',4,'y2',4);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% hold off;  
intersect = calling_function(line1,line2)
```

```
intersect = logical  
0
```

Example 8

correct output: true

```
line1 = struct('x1',0,'y1',0,'x2',3,'y2',3);  
line2 = struct('x1',2,'y1',2,'x2',4,'y2',4);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% hold off;
```

```
intersect = calling_function(line1,line2)
```

```
intersect = logical  
1
```

Example 9

correct output: false

```
line1 = struct('x1',1,'y1',2,'x2',3,'y2',2);  
line2 = struct('x1',0,'y1',4,'x2',5,'y2',4);  
  
% For a visual of the lines uncomment the code below  
% figure  
% hold on;  
% plot([line1.x1,line1.x2],[line1.y1,line1.y2],'-o')  
% plot([line2.x1,line2.x2],[line2.y1,line2.y2],'-o')  
% axis([-1 5 -1 5])  
% hold off;  
intersect = calling_function(line1,line2)
```

```
intersect = logical  
0
```

HW8P4_do_polygon_edges_intersect

```
program_name = "HW8P4_do_polygon_edges_intersect";  
calling_function = str2func(program_name + lastname);
```

Example 1

correct output: false

```
vertices(1:4) = struct('x',[],'y',[]);  
  
[vertices.x] = deal(10,30,20,38);
```

Error using deal
The number of outputs should match
the number of inputs.

```
[vertices.y] = deal(10,12,20,15);  
  
edges_intersect = calling_function(vertices,false)
```

Example 2

correct output: true

```
edges_intersect = calling_function(vertices,true)
```

Example 3

correct output: true

```
vertices(1:7) = struct('x',[],'y',[]);  
  
[vertices.x] = deal(5,30,2,3,12,7,16);  
[vertices.y] = deal(10,22,0,15,11,3,33);  
  
edges_intersect = calling_function(vertices,false)
```

Example 4

correct output: true

```
edges_intersect = calling_function(vertices,true)
```